

Security Monitoring & Detection Engineering

Table of Contents

1. Log Management
2. Security Tools
3. Log Analysis
4. Document Security Events
5. Monitoring Dashboards
6. Key Learnings
7. Conclusion
8. References

Log Management

1. Log Collection Pipeline

Set up Fluentd on Ubuntu to collect Syslog. Test by generating logs with logger "Test message". Forward logs to Elastic SIEM and verify receipt

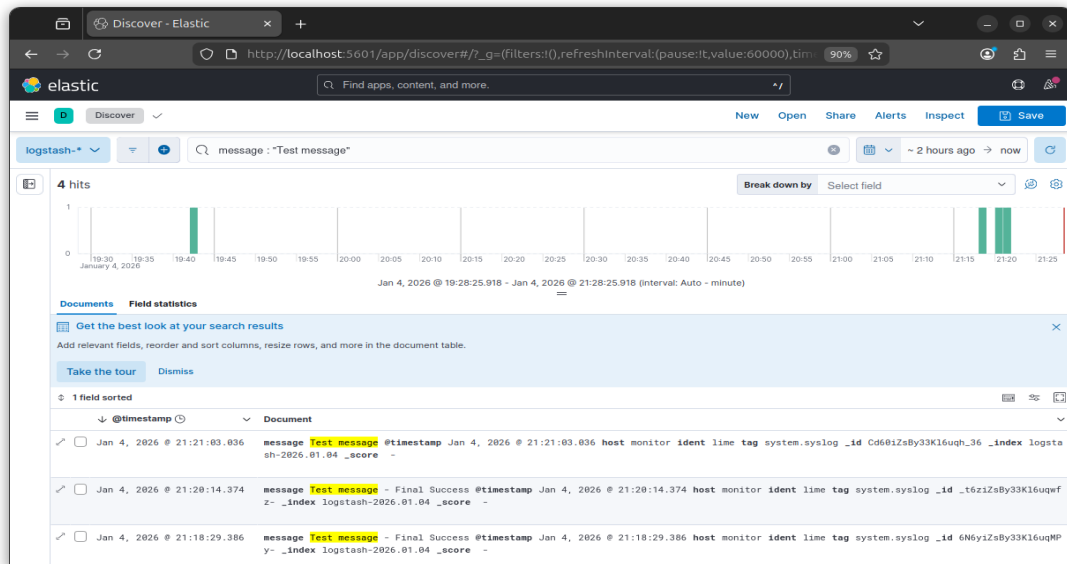


Figure 1: Fluentd to Elastic SIEM

2. KQL Query Practice

In Elastic SIEM, write a KQL query to find Event ID 4625

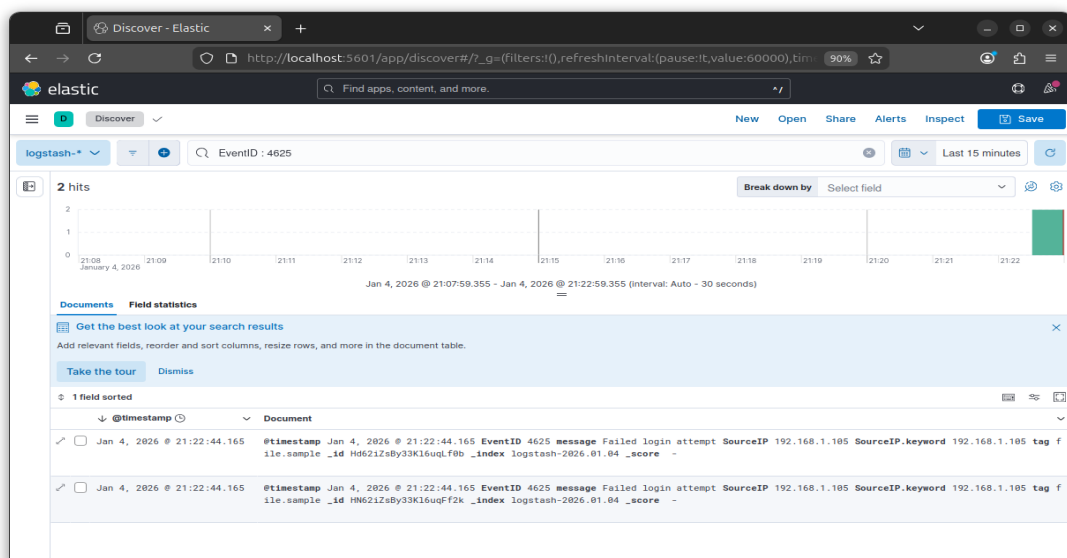


Figure 2: KQL Query for Audit Failure

3. Normalization Exercise

Convert an Apache access log to JSON. Save output to a file and check the format

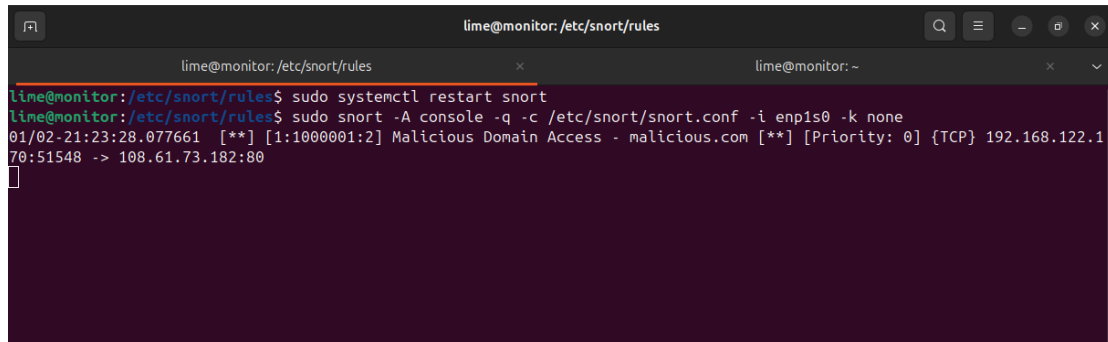
```
zeltij (elasticstack) > elasticserver fleetserver ubuntu@elasticagent  
elkuser@elkserver: ~$ tail -n 30 apache_logs.json  
  
    },  
    "version": "1.1"  
  },  
  "event": {  
    "agent_id.status": "verified",  
    "ingested": "2026-01-23T16:57:24Z",  
    "kind": "event",  
    "created": "2026-01-23T16:57:14.390Z",  
    "category": [  
      "web"  
    ],  
    "dataset": "apache.access",  
    "outcome": "success"  
  },  
  "user_agent": {  
    "original": "Elastic-Metricbeat/9.2.4 (Linux; amd64; fd989e2bd4416ce14162971875d6013334f6fd44; 2026-01-08 19:21:12 +0000 UTC)",  
    "os": {  
      "name": "Linux"  
    },  
    "name": "Other",  
    "device": {  
      "name": "Other"  
    }  
  }  
}  
]  
}  
} ]
```

Figure 3: Raw Apache Access Logs to JSON

Security Tools

1. Snort Rule Testing

Write a rule to detect HTTP requests to "malicious.com":



```
lime@monitor: /etc/snort/rules
lime@monitor: /etc/snort/rules$ sudo systemctl restart snort
lime@monitor: /etc/snort/rules$ sudo snort -A console -q -c /etc/snort/snort.conf -i enp1s0 -k none
01/02-21:23:28.077661  [**] [1:1000001:2] Malicious Domain Access - malicious.com [**] [Priority: 0] {TCP} 192.168.122.1
70:51548 -> 108.61.73.182:80
```

Figure 4: Snort Rule for Malicious Domain

2. Nessus Scan

Run Nessus Essentials against Metasploitable2 and list the top 3 vulnerabilities by CVSS score

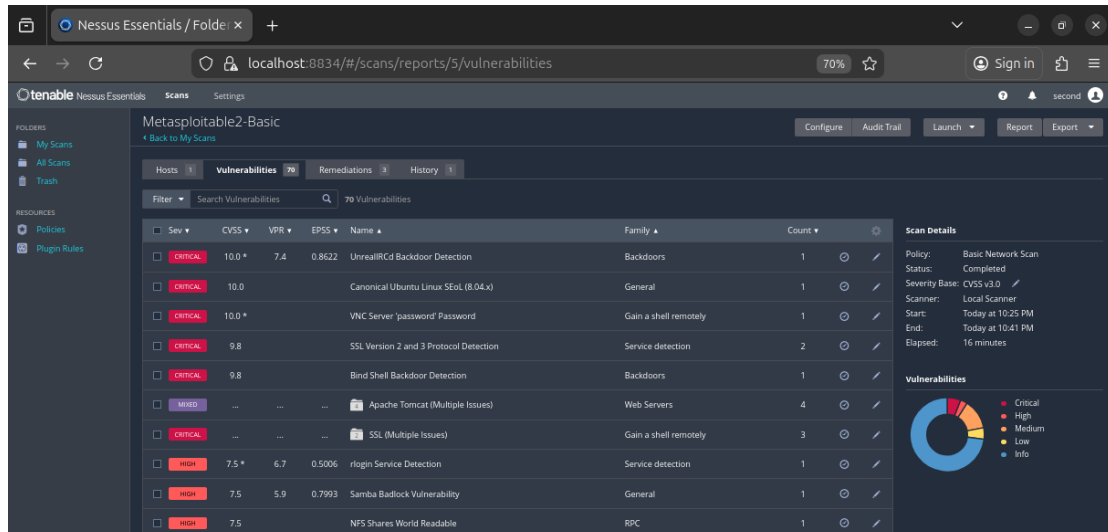
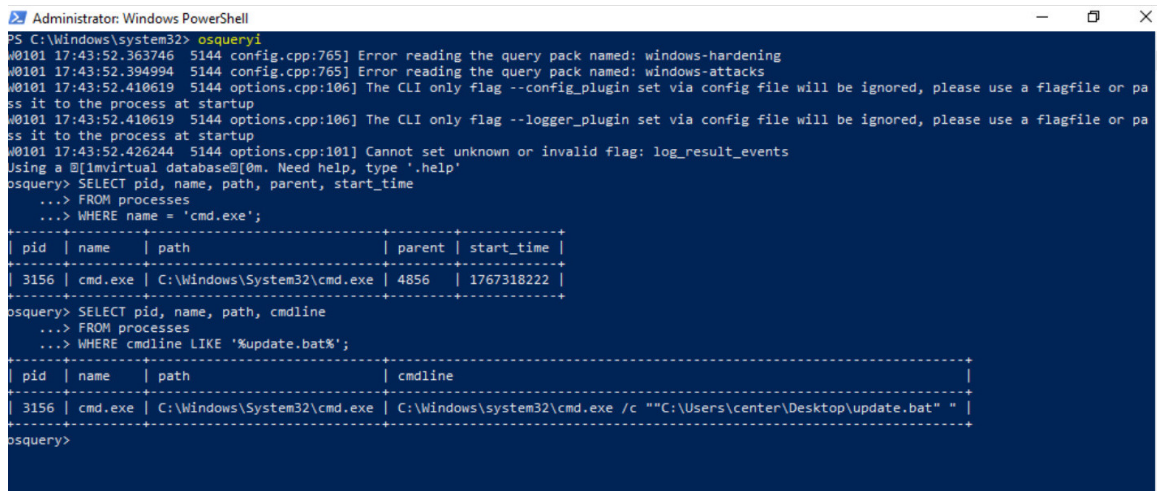


Figure 5: Nessus Vulnerability Scan Results

3. Osquery Monitoring

Install Osquery on Windows VM. Query running processes (SELECT * FROM processes;) and simulate a malicious process with a harmless batch file



```
Administrator: Windows PowerShell
PS C:\Windows\system32> osqueryi
[0101 17:43:52.363746 5144 config.cpp:765] Error reading the query pack named: windows-hardening
[0101 17:43:52.394994 5144 config.cpp:765] Error reading the query pack named: windows-attacks
[0101 17:43:52.410619 5144 options.cpp:106] The CLI only flag --config_plugin set via config file will be ignored, please use a flagfile or pass it to the process at startup
[0101 17:43:52.410619 5144 options.cpp:106] The CLI only flag --logger_plugin set via config file will be ignored, please use a flagfile or pass it to the process at startup
[0101 17:43:52.426244 5144 options.cpp:101] Cannot set unknown or invalid flag: log_result_events
Using a @[1]mvirtual database@[0]m. Need help, type '.help'
osquery> SELECT pid, name, path, parent, start_time
...> FROM processes
...> WHERE name = 'cmd.exe';
+-----+-----+-----+-----+-----+
| pid | name | path | parent | start_time |
+-----+-----+-----+-----+-----+
| 3156 | cmd.exe | C:\Windows\System32\cmd.exe | 4856 | 1767318222 |
+-----+-----+-----+-----+-----+
osquery> SELECT pid, name, path, cmdline
...> FROM processes
...> WHERE cmdline LIKE '%update.bat%';
+-----+-----+-----+-----+
| pid | name | path | cmdline |
+-----+-----+-----+-----+
| 3156 | cmd.exe | C:\Windows\System32\cmd.exe | C:\Windows\system32\cmd.exe /c ""C:\Users\center\Desktop\update.bat" "" |
+-----+-----+-----+-----+
osquery>
```

Figure 6: Querying System Processes via Osquery

Log Analysis Practice

1. Windows Event Viewer

Filter for Event ID 4625 or 7045

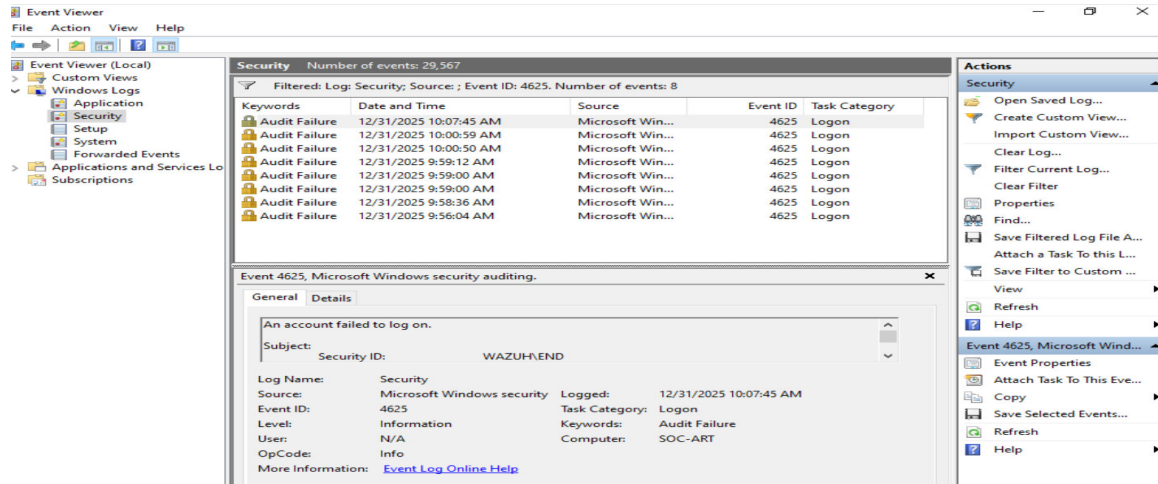


Figure 7: Filtering for Authentication Failures

2. Zimmerman Tools

Use Eric Zimmerman's Tools to parse Windows Shortcut (LNK) files for malicious content

```
Processing C:\Users\center\Desktop\TestSite.lnk
Source file: C:\Users\center\Desktop\TestSite.lnk
Source created: 2026-01-11 05:40:02
Source modified: 2026-01-11 05:42:20
Source accessed: 2026-01-11 05:56:16

--- Header ---
Target created: 2026-01-11 05:40:01
Target modified: 2026-01-11 05:41:50
Target accessed: 2026-01-11 05:41:50

File size (bytes): 42
Flags: HasTargetIdList, HasLinkInfo, HasRelativePath, IsUnicode
File attributes: FileAttributeArchive
Icon index: 0
Show window: SwNormal (Activates and displays the window. The window is restored to its original size and position if the window is minimized or maximized.)

Relative Path: ..\Downloads\test_evidence.txt

--- Link information ---
Flags: VolumeIdAndLocalBasePath

>> Volume information
Drive type: Fixed storage media (Hard drive)
Serial number: 9422F26F
Label: (no label)
Local path: C:\Users\center\Downloads\test_evidence.txt

--- Target ID information (Format: Type ==> Value) ---
Absolute path: This PC\Downloads\test_evidence.txt
-Root folder: GUID ==> This PC
-Root folder: GUID ==> Downloads
-File ==> test_evidence.txt
Short name: TEST_E-1.TXT
Modified: 2026-01-11 05:41:52
Extension block count: 1

----- Block 0 (BeeF0004) -----
Long name: test_evidence.txt
Created: 2026-01-11 05:40:02
Last access: 2026-01-11 05:41:52
MFT entry/sequence #: 71274/2 (0x1166A/0x2)
```

Figure 8: Parsing LNK Files for Malicious Indicators

Document Security Events

1. Mock Event

Create template with Date/Time | Source IP | Event ID | Description | Action Taken

B26						
	A	B	C	D	E	F
1	Date/Time (UTC)	Source IP	Event ID	Description	Action Taken	
2	2026-01-11 09:42	192.168.1.10	AUTH-FAIL-1001	Multiple failed login attempts against admin account	IP blocked; password reset; SOC Tier 2 notified	
3	2026-01-11 10:05	192.168.1.15	AUTH-SUCCESS-2001	Successful login after multiple failures from same IP	Account monitored; user contacted for verification	
4	2026-01-11 10:33	10.0.0.23	PORT-SCAN-3002	Port scanning activity detected targeting web server	Source IP blocked; IDS alert reviewed	
5	2026-01-11 11:02	172.16.5.44	MALWARE-DET-4007	Endpoint malware detection: suspicious PowerShell execution	Endpoint isolated; malware removed; forensic analysis started	
6	2026-01-11 11:47	203.0.113.77	BRUTEFORCE-5003	Brute-force SSH attempts detected (45 attempts)	Firewall rule applied; SSH keys rotated	
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						

Figure 9: Mock Event Documentation

Monitoring Dashboards

1. Top 10 source IPs generating alerts

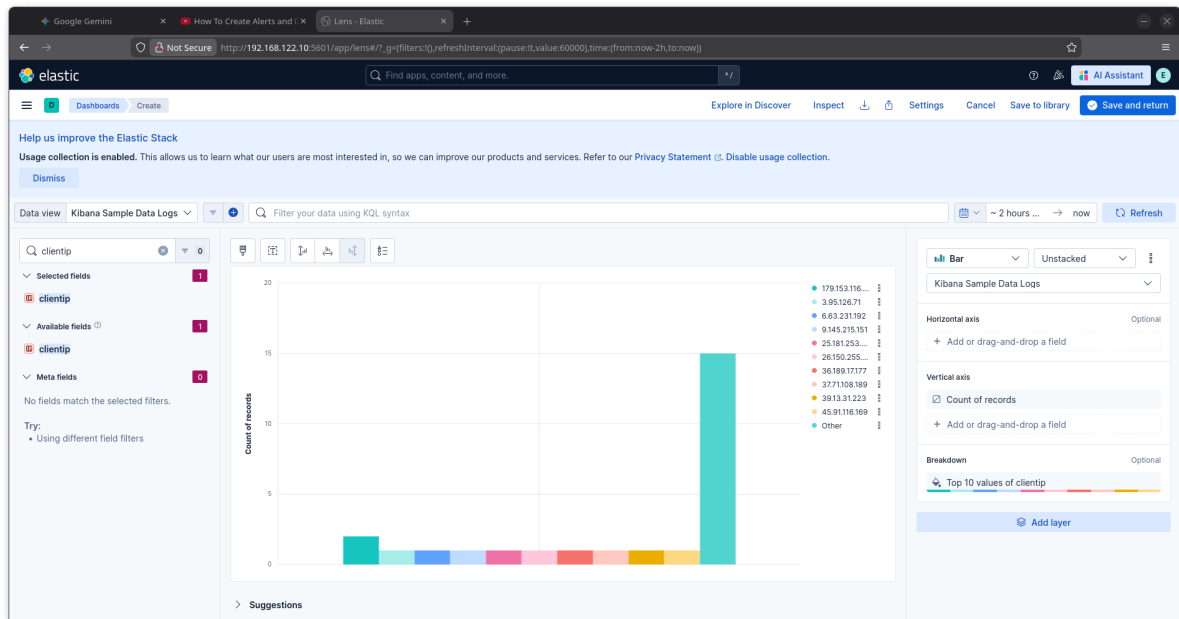


Figure 10: Top 10 Source IP Distribution

2. Frequency of critical Event Ids

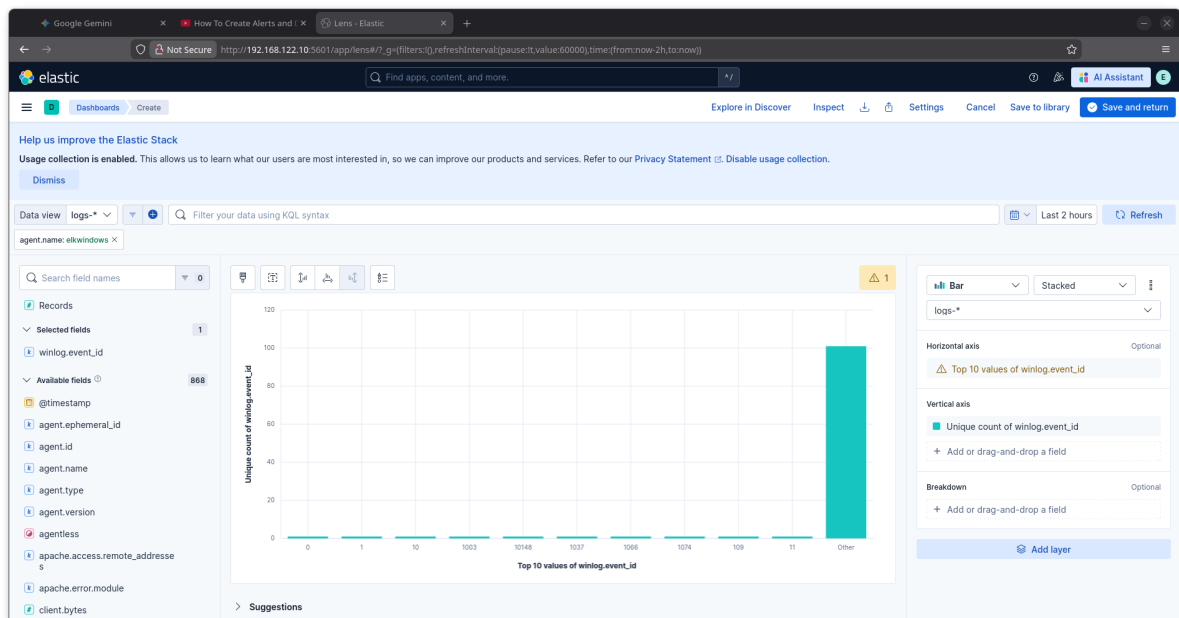


Figure 11: Top 10 Event ID Distribution

3. Prebuilt Dashboard

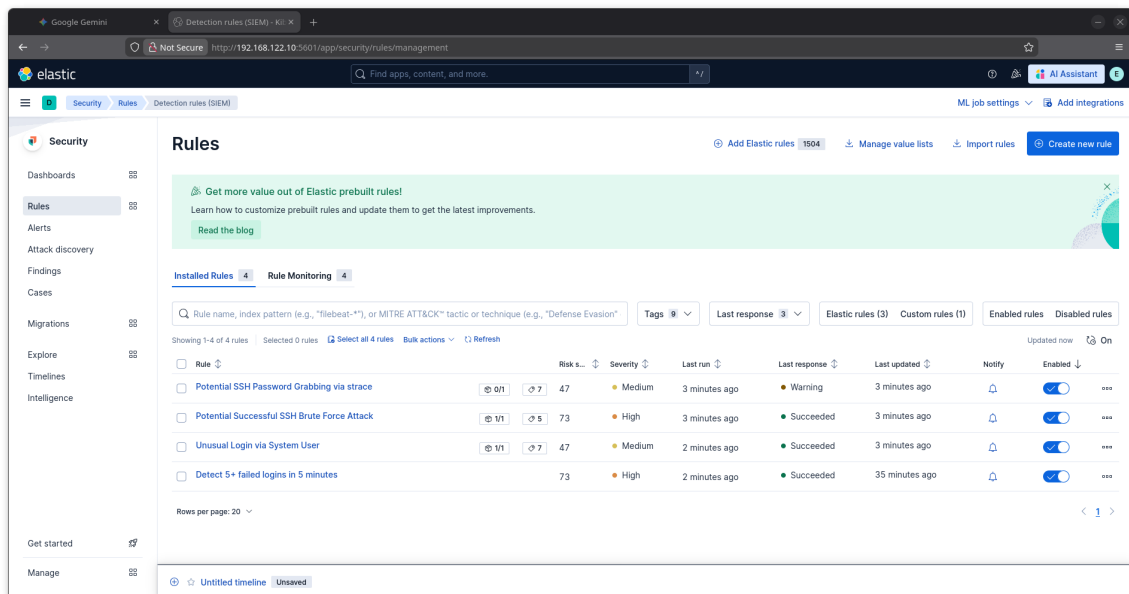


Figure 12: Sigma-Based Behavioral Analytics

4. Configure Alert Rules

In Elastic SIEM: Create: Rule: "Detect 5+ failed logins in 5 minutes", Index: security- login-*, Condition: count > 5

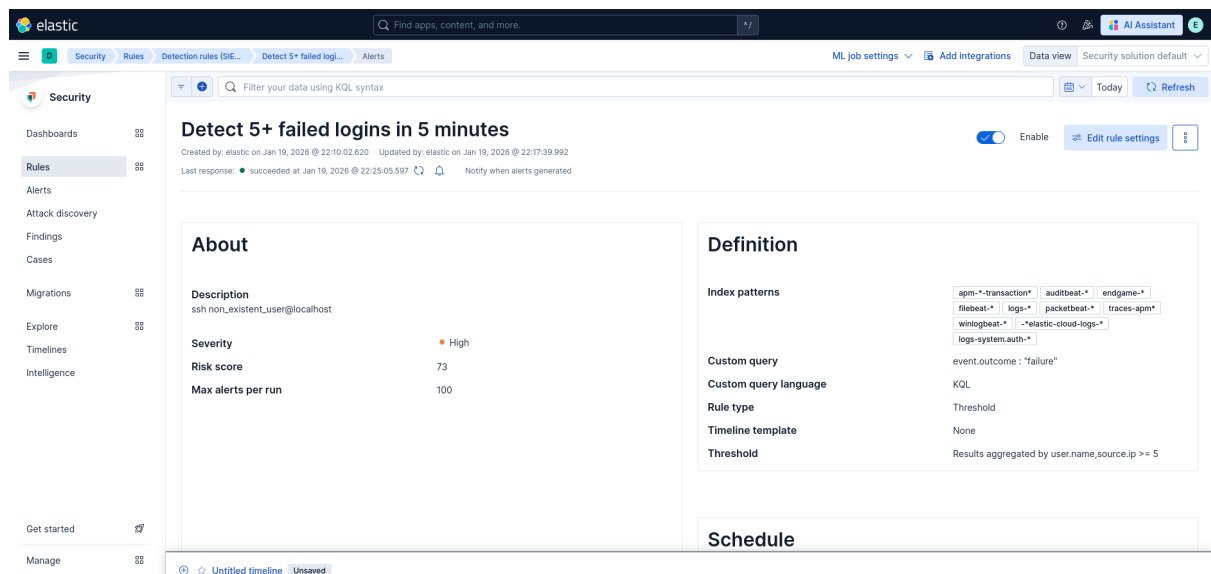


Figure 11: Failed Login Threshold Rule

5. Alert Rule in Wazuh

Create a rule in Wazuh to detect 3+ failed logins in 2 minutes

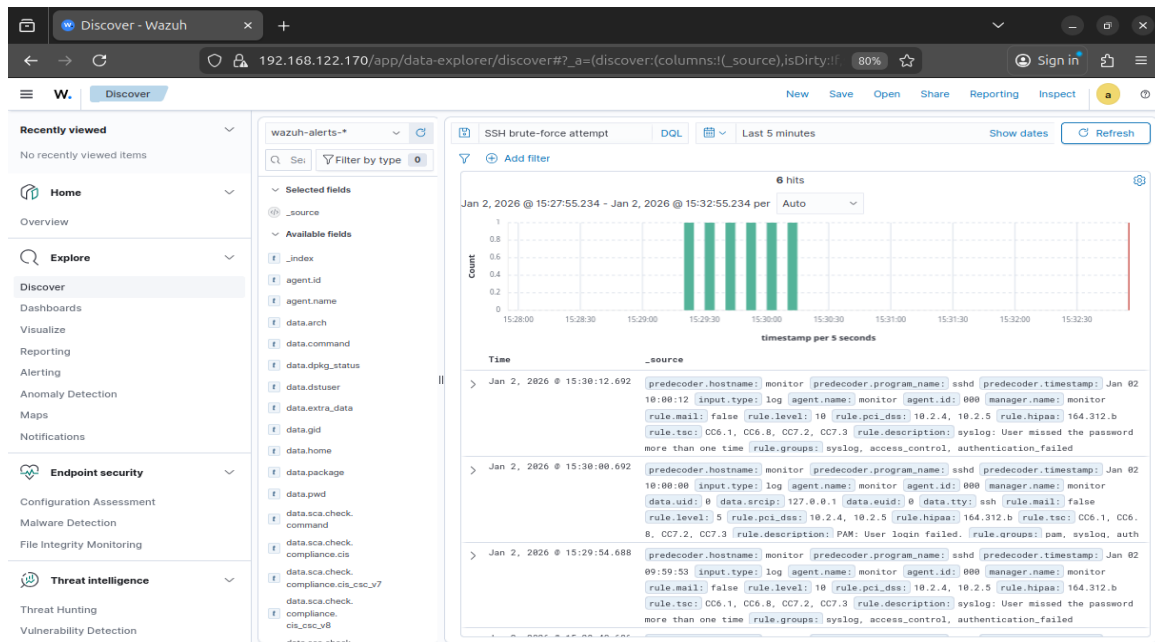


Figure 12: Wazuh Brute-Force Detection Rule

Key Learnings

1. Integration of Automated Detection Engineering: Transitioning from manual log review to automated alerting is a fundamental requirement for modern security operations. By implementing **Sigma rules** and custom **Wazuh/Elastic threshold logic**, the project successfully transformed raw telemetry into actionable intelligence. This implementation ensures that high-frequency threats, such as SSH brute-force attacks (Event ID 4625), are flagged in real-time, significantly reducing the mean time to detect (MTTD) without requiring constant human oversight.

2. Forensic Visibility through Structured Telemetry: Visibility serves as the primary foundation for defensive operations. Utilizing **Osquery** for Windows endpoint monitoring and **Zimmerman's LECmd** for LNK file analysis provided deep visibility into system-level activity. This approach demonstrates a commitment to "forensic readiness," moving beyond simple log collection to the parsing of complex artifacts. These methods are essential for identifying indicators of malicious persistence and lateral movement that traditional logging might overlook.

3. Operational Efficiency through Data Transformation: Effective security monitoring is predicated on standardized, machine-readable data. The conversion of **Apache logs to JSON** and the subsequent development of **Kibana/Grafana dashboards** highlight the critical role of data normalization. Structured data enables the high-speed, complex visualizations—such as Top 10 Source IP distributions and event frequency heatmaps—necessary for identifying anomalies and emerging threats at scale.

Conclusion

The project moved beyond passive observation by deploying and testing active security controls. By writing custom Snort IDS rules to flag malicious domains and engineering Wazuh/Elastic threshold alerts to detect brute-force patterns (Event ID 4625), the environment was proven capable of identifying threats in real-time. The integration of Nessus vulnerability scanning and Osquery process monitoring ensured a defense-in-depth posture, allowing for a full security operations workflow from initial detection and triage to documented incident response.

References

1. Elastic Stack (Kibana/SIEM): <https://www.elastic.co/docs>
2. Wazuh (Open Source XDR): <https://documentation.wazuh.com/>
3. Snort IDS/IPS: <https://www.snort.org/documents>
4. Osquery: <https://osquery.io/>
5. Eric Zimmerman's Tools: <https://ericzimmerman.github.io/>
6. Nessus Essentials:
<https://www.tenable.com/products/nessus/nessus-essentials>
7. Fluentd: <https://docs.fluentd.org/>
8. Logstash:
<https://www.elastic.co/guide/en/logstash/current/index.html>
9. Metasploitable2: <https://sourceforge.net/projects/metasploitable/>